

Day 10 — Combinational Logic Mastery

Target time: 90–120 minutes

Suggested rhythm: 35 minutes derive · 35 minutes use the Logic Gate Lab · 20 minutes draw and explain · 10–30 minutes repair weak spots

A circuit can choose, compare, decode, and add. Yet after its inputs return to their old values, it has no evidence that anything happened.

The question for today

You already know the six common gates, truth tables, Boolean expressions, De Morgan's laws, and why NAND and NOR are universal. Today is not another tour of gate names.

Today we ask:

What becomes possible when gates are composed into a network—and what remains impossible until we add memory?

The answer is **combinational logic**: circuits whose settled outputs are determined entirely by their current inputs.

current inputs — physical gate network — settled outputs
A,B,C topology Y,Z

This simple rule gives us selection, decoding, comparison, and arithmetic. It is the beginning of a CPU datapath. It also draws a hard boundary: the network cannot remember yesterday, one clock ago, or even one transition ago.

By the end, you should be able to explain how a physical circuit implements a Boolean function without understanding it, then support that explanation with four kinds of evidence: **Explain, Draw, Observe, Build**.

1. One function, three descriptions

Consider:

$$Y = (A \wedge B) \vee (\neg A \wedge C)$$

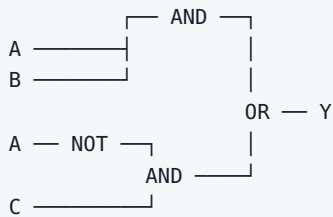
The expression is compact, but it is not the only useful view.

Expression: the symbolic rule

$$Y = (A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C)$$

It tells us which operations compose and how they are grouped.

Gate network: one physical structure



Wires carry intermediate states. Each gate responds only to its local inputs.

Truth table: complete settled behavior

Name intermediate values instead of guessing:

$P = A \wedge B$
 $Q = \neg A$
 $R = Q \wedge C$
 $Y = P \vee R$

A	B	C	$P=A \wedge B$	$Q=\neg A$	$R=Q \wedge C$	$Y=P \vee R$
0	0	0	0	1	0	0
0	0	1	0	1	1	1
0	1	0	0	1	0	0
0	1	1	0	1	1	1
1	0	0	0	0	0	0
1	0	1	0	0	0	0
1	1	0	1	0	0	1
1	1	1	1	0	0	1

Look at the final column another way:

when $A=0$, Y follows C
when $A=1$, Y follows B

We have accidentally built something important: a **2-to-1 multiplexer**. A is the select input. The circuit chooses one of two data inputs.

This is a recurring engineering pattern:

behavioral need $\rightarrow$ Boolean rule $\rightarrow$ gate network $\rightarrow$ physical timing and cost

Pause and derive

Cover the table. Rebuild all eight rows from the intermediate expressions. Then explain why the circuit selects C when $A=0$ and B when $A=1$.

2. Input count determines truth-table size

Each independent Boolean input doubles the number of possible input combinations.

```
1 input → 21 = 2 rows
2 inputs → 22 = 4 rows
3 inputs → 23 = 8 rows
4 inputs → 24 = 16 rows
n inputs → 2n rows
```

Why multiplication rather than addition? For every existing pattern, a new input can be either 0 or 1:

```
patterns for A,B:      00  01  10  11
append C=0 or C=1:    000 001 010 011 100 101 110 111
```

Outputs do not change the number of rows. A circuit with 3 inputs and 20 outputs still has 8 possible input combinations; each row simply records 20 output bits.

This growth explains why truth tables are excellent for small circuits but not sufficient as a practical design document for an entire CPU. A function of 64 independent input bits has:

```
264 = 18,446,744,073,709,551,616 input combinations
```

Designers use hierarchy, algebra, hardware-description languages, simulation, formal methods, and repeated structures. The truth-table idea remains the behavioral foundation even when nobody writes every row.

Quick check

Without calculating mechanically:

1. How many rows does a five-input truth table have?
2. If one of those inputs is permanently tied to 0, how many combinations remain reachable?
3. Why does adding three output wires not create more input rows?

3. Combinational means no intentional memory

For a combinational circuit:

```
outputs = f(current inputs)
```

Suppose the inputs are `A=0, B=1, C=1`. Once the signals settle, the function fixes every output. If the inputs change and later return to `0,1,1`, the outputs also return to the same values.

There is no variable named “previous A” hidden inside an AND gate.

The memory test

Ask this of any proposed circuit:

Can two visits to the same current input pattern produce different stable outputs because of what happened earlier?

If yes, there is state somewhere. The system is not purely combinational.

Pure combinational logic cannot:

- count how many pulses arrived;
- remember that an error occurred;
- preserve a result after its source inputs disappear;
- know which instruction came before this one;
- advance through a sequence by itself.

That requires storage, feedback, and usually controlled timing. Those belong to the later sequential-logic story.

A subtle distinction

A physical gate has capacitance and takes time to change. That does **not** make ordinary gate delay useful architectural memory. Residual charge and transient behavior are physical realities, but a combinational design does not promise that software or another block can rely on them as stored state.

The abstraction is:

ignore transition details for a moment
reason about the stable value after the network settles

4. Real gates do not update instantaneously

The truth table says what value is correct after settling. It says nothing about how quickly that value appears.

Every transistor and wire has finite physical behavior. A gate needs time for its output voltage to respond to a changed input. This is **propagation delay**.

Consider two paths:



In ideal Boolean algebra, $A \oplus \neg A$ is always 1. But one path contains an extra inverter. When A changes, the direct and inverted signals do not arrive together. For a short interval, both XOR inputs might momentarily look equal, making Y pulse to 0.

That unintended transient is a **glitch**. More generally, a **hazard** is a circuit structure capable of producing an unwanted transition because different paths have different delays.

ideal settled story: Y ————— HIGH

possible physical story: Y ———┐——— HIGH

└───┘

brief glitch

This is only a preview. Do not turn today into a timing-analysis course. Keep two truths separate:

1. Boolean analysis determines the correct settled output.
2. Timing analysis asks when the output becomes valid and what happens on the way there.

A CPU clock period, pipeline design, and safe sampling all depend on the second question. The longest relevant path through a combinational block is often called its **critical path**.

5. Decoder intuition: an encoded choice becomes one active line

Two bits can encode four choices:

00, 01, 10, 11

A 2-to-4 decoder turns that compact code into four output lines, exactly one of which is active:

A	B	D0	D1	D2	D3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Derive each line:

D0 = $\neg A \wedge \neg B$
D1 = $\neg A \wedge B$
D2 = $A \wedge \neg B$
D3 = $A \wedge B$

The decoder has not learned what “choice 2” means. Its topology merely makes one conjunction true for each input pattern.

Where does this intuition reappear?

- selecting one register;
- enabling one device or memory region;
- decoding part of an instruction;
- identifying one operation or destination.

Real decoders often include an enable input and may use active-low outputs, but the central idea remains: **compact binary code in, one selected line out**.

6. Multiplexer intuition: many candidates, one path onward

A multiplexer, or **mux**, solves the reverse-shaped problem:

several candidate values + select bits → one chosen output

For two data inputs:

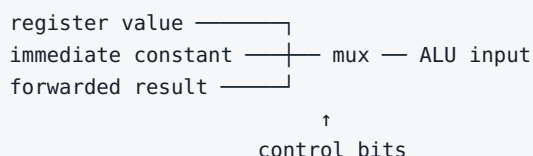
S	D0	D1	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

The compact expression is:

$$Y = (\neg S \wedge D0) \vee (S \wedge D1)$$

When $S=0$, the first term can pass $D0$ and the second is forced to 0. When $S=1$, the reverse happens.

Muxes are everywhere in a datapath:



“Choose this source” becomes a physical network of gates. The mux does not understand registers, constants, or forwarding. It only maps select and data bits to output bits.

7. Comparator intuition: ask a relation bit by bit

For one bit, equality is simply “the inputs do not differ”:

$$\text{equal}(A,B) = \neg(A \oplus B) \quad (\text{XNOR})$$

For two two-bit words $A1A0$ and $B1B0$, every corresponding pair must match:

$$EQ = \neg(A1 \oplus B1) \wedge \neg(A0 \oplus B0)$$

This scales naturally across a wider word.

Ordering is more structured. For unsigned two-bit values:

A > B when:
A1 is 1 and B1 is 0
OR
the high bits are equal and A0 is 1 and B0 is 0

The high bit gets priority because it has greater place value.

Comparators feed branch decisions, bounds checks, address matching, cache tags, and status flags. Signed comparison needs interpretation rules for the sign bit; today the important insight is that “equal” and “greater than” can be decomposed into gate-level relations.

8. Derive the half-adder—do not memorize it

We will touch addition only far enough to prove that arithmetic emerges from Boolean functions.

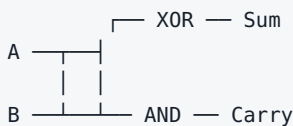
Add two one-bit inputs, A and B:

A	B	ordinary total	two-bit result	Sum	Carry
0	0	0	00	0	0
0	1	1	01	1	0
1	0	1	01	1	0
1	1	2	10	0	1

Now compare the output columns with gate behavior:

Sum is 1 exactly when A and B differ:
Sum = A XOR B

Carry is 1 exactly when both A and B are 1:
Carry = A AND B



That circuit is a **half-adder**.

Why “half”? It has no carry-in from a less-significant position. We are not building a full multi-bit adder today. Preserve these open questions:

- How is a carry-in included?
- How are one-bit stages connected?
- How quickly can a carry cross a wide word?
- Where are operands and results held?

Those questions lead into adders, timing, and storage later. Today's evidence is narrower and more important: a numerical operation can be specified as output bits and realized as composed Boolean functions.

9. De Morgan and universal gates are structural tools

You have already met:

$$\neg(A \wedge B) = \neg A \vee \neg B$$
$$\neg(A \vee B) = \neg A \wedge \neg B$$

Today, view them as permission to change structure without changing settled behavior.

Using only NAND:

```
NOT A = NAND(A,A)

P = NAND(A,B)
AND(A,B) = NAND(P,P)

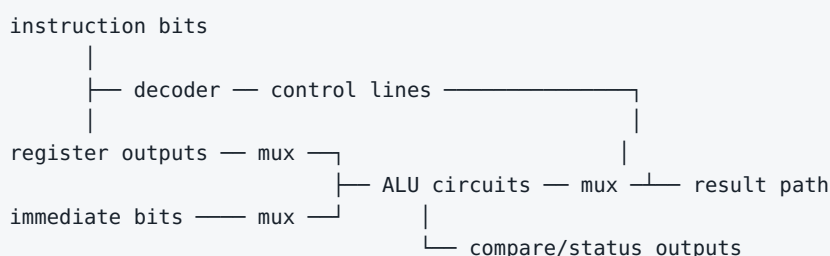
NA = NAND(A,A)
NB = NAND(B,B)
OR(A,B) = NAND(NA,NB)
```

Different topology, same truth table. That equivalence matters because real hardware design balances gate availability, transistor count, power, layout, fan-out, and delay—not just algebraic elegance.

In the lab, do not merely observe that two results match for one input. Test all four input rows. Equivalence over a two-input Boolean domain means matching on every possible row.

10. From composed gates to the CPU datapath

Put the pieces together:



An ALU may contain parallel networks for AND, OR, XOR, shifts, comparison, and arithmetic. Control signals select which result continues.



The surrounding **datapath** is the collection of routes and functional blocks through which data moves: registers, muxes, ALU, buses, and related connections. Control signals steer those routes.

Preserve the layers:

Boolean function	describes an input/output mapping
gate network	implements that mapping structurally
transistors/wires	produce the physical voltage behavior
datapath	composes useful transformations and routes
ISA/software	assigns higher-level meaning to the bit patterns

An equality comparator inside a CPU does not know it is checking a loop condition. A decoder does not understand an instruction. A mux does not know why one operand matters. Each responds locally; meaningful behavior emerges from composition and the interpretation imposed by the larger system.

11. Logic Gate Lab

Open:

`daily/challenges/day-010-logic-gate-lab.html`

Use this loop:

1. **Predict** before changing a switch or choosing an answer.
2. **Observe** the selected truth-table row and intermediate signals.
3. **Explain** the output from the Boolean rule.
4. **Build** evidence by completing each challenge family.

Suggested route:

- 10 minutes: gate bench and live truth table;
- 10 minutes: expression explorer and row-count intuition;
- 10 minutes: De Morgan and NAND-only constructions;
- 15 minutes: half-adder prediction challenge;
- 10 minutes: transfer challenge across mux, decoder, and comparator;
- 5 minutes: speak the mastery answer aloud.

Challenge explanations remain hidden until you commit an attempt. A score is diagnostic evidence, not proof of understanding.

12. Mastery checkpoint

Close the lesson and answer:

How can a physical circuit implement a Boolean function without understanding it?

A strong answer should form one causal chain:

```
physical input voltages
  ↓ interpreted by engineered LOW/HIGH ranges
transistors respond locally to electric fields
  ↓
network topology permits or blocks conducting paths
  ↓
output is driven toward a physical voltage range
  ↓ interpreted as 0 or 1
input/output mapping matches a Boolean truth table
```

Nothing in that chain needs the application-level meaning of the bits. “Boolean function” is our abstract description of the stable mapping produced by the physical arrangement.

Use one concrete example. Series contacts make an AND path: current can reach the output only if both contacts close. The contacts do not evaluate the English word “and.” Their arrangement causes behavior that our truth table describes as AND.

Then extend the answer: composed networks implement selection, decoding, comparison, and addition by the same principle. A CPU gains useful behavior from enormous structured composition, not from gates individually understanding instructions, numbers, or programs.

13. Evidence: Explain, Draw, Observe, Build

Explain

Without notes:

- define a combinational circuit;
- derive why n inputs require 2^n truth-table rows;
- explain why propagation delay does not make a combinational block a memory;
- distinguish settled Boolean behavior from transition timing;
- answer the mastery checkpoint with a physical example.

Draw

On one page, draw:

```
two-bit code → 2-to-4 decoder

D0,D1 + S → 2-to-1 mux → Y

A,B → half-adder → Sum,Carry

gate networks → ALU/result mux → CPU datapath
```

Label intermediate Boolean expressions. Add one possible unequal-delay path and mark where a glitch could appear.

Observe

In the HTML lab:

- test all rows of at least three gates;
- make each De Morgan pair match across all four rows;

- observe NAND-only NOT, AND, and OR;
- complete the half-adder prediction set before reading explanations;
- record one prediction that changed after observation.

Build

On paper or in the expression explorer:

1. Build $Y = (\neg S \wedge D0) \vee (S \wedge D1)$.
2. Build $EQ = \neg(A \oplus B)$ for one-bit equality.
3. Build half-adder Sum and Carry.
4. Rebuild OR using only NAND.
5. Write the full truth table for one construction and show that it matches the intended function.

Do not mark the checkpoint complete merely because the lab reports a high score. The evidence is the combination of prediction, derivation, drawing, observation, and explanation.

14. Final retrieval

Fill these blanks from memory:

combinational output = $f(\rule{1.5cm}{0.4pt})$

n independent Boolean inputs produce $\rule{1cm}{0.4pt}$ truth-table rows

a decoder turns an encoded choice into $\rule{2cm}{0.4pt}$

a mux chooses $\rule{2cm}{0.4pt}$

one-bit equality is NOT($\rule{1cm}{0.4pt}$)

half-adder Sum = $\rule{1cm}{0.4pt}$ Carry = $\rule{1cm}{0.4pt}$

combinational logic cannot $\rule{2cm}{0.4pt}$

Then finish this sentence in your own words:

A physical circuit implements a Boolean function without understanding it because ...

If that final explanation connects voltage, local transistor behavior, topology, truth-table mapping, and absence of semantic understanding, today's central model is in place.

References used selectively

- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems*, Chapter 3, sections on combinational logic, logical completeness, decoders, multiplexers, and adders.
- Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*, chapters that build switching logic into arithmetic and data selection.

- The prior Boolean-logic lesson and offline gate lab were used as the dense retrieval base for gates, truth tables, De Morgan's laws, universal gates, physical-switch intuition, and the half-adder preview.

Next bridge: combinational blocks can transform values, but a CPU must also preserve operands, results, and control state. That requires a new physical idea: controlled memory.